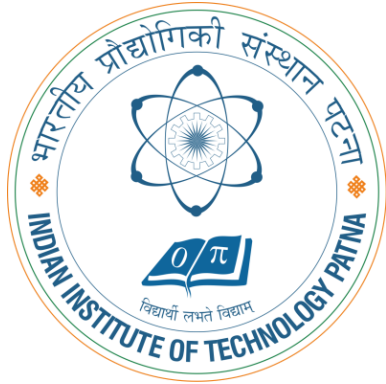


CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

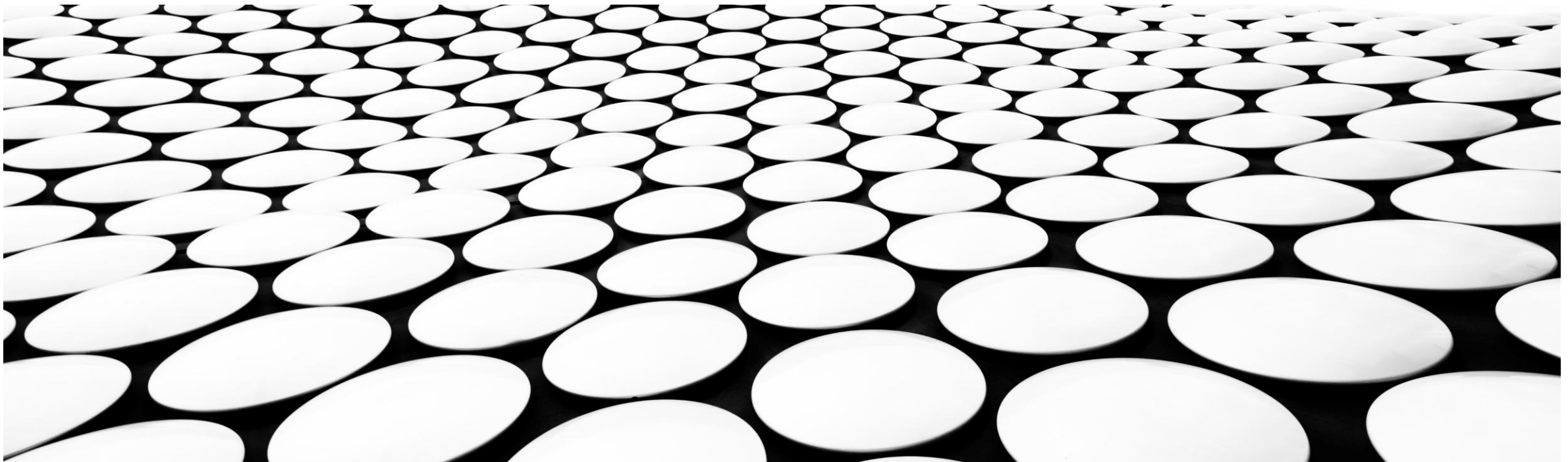


LECTURE 13: CONTROL UNIT DESIGN IV

DR. ARIJIT ROY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

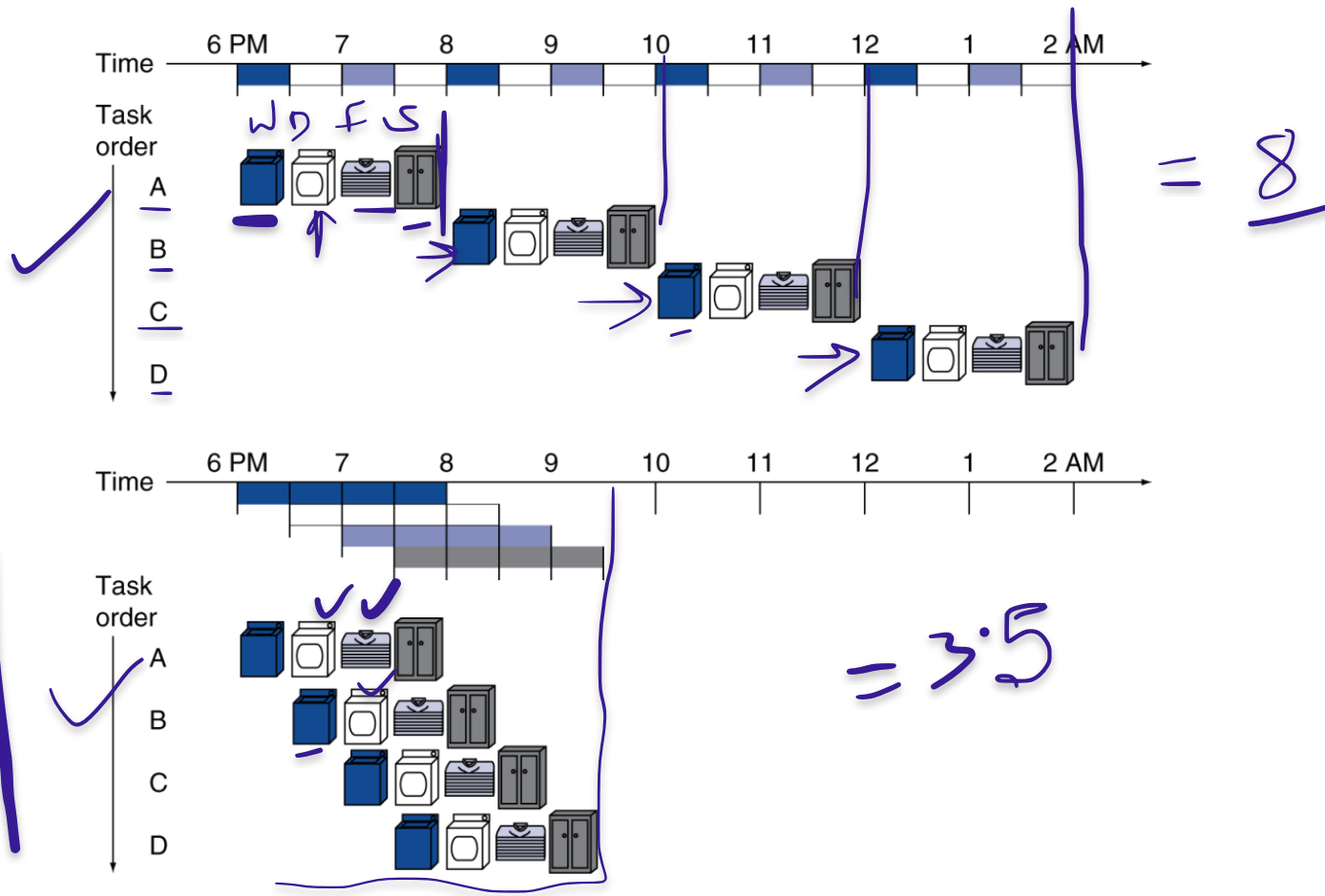
INDIAN INSTITUTE OF TECHNOLOGY PATNA



RECAP OF PIPELINED PROCESSOR

PIPELINING ANALOGY

- Pipelined laundry: overlapping execution
- Parallelism improves performance



Pipelining is an implementation technique in which multiple instructions are overlapped in execution

Four loads:

Speedup
 $= 8/3.5 = 2.3$

PIPELINED MIPS PROCESSOR

- Temporal parallelism
- Divide single-cycle processor into 5 stages:
 - Fetch ✓
 - Decode ✓
 - Execute ✓
 - Memory ✓
 - Writeback ✓
- Add **pipeline registers** between stages

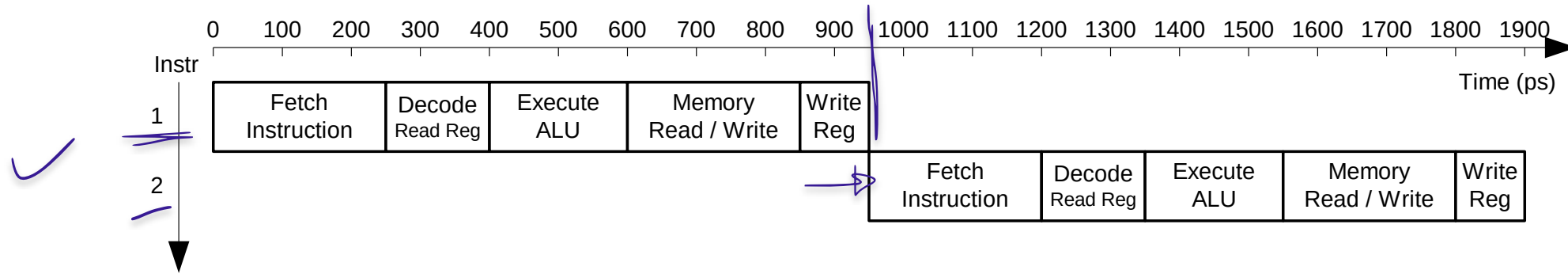
PIPELINE PERFORMANCE

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

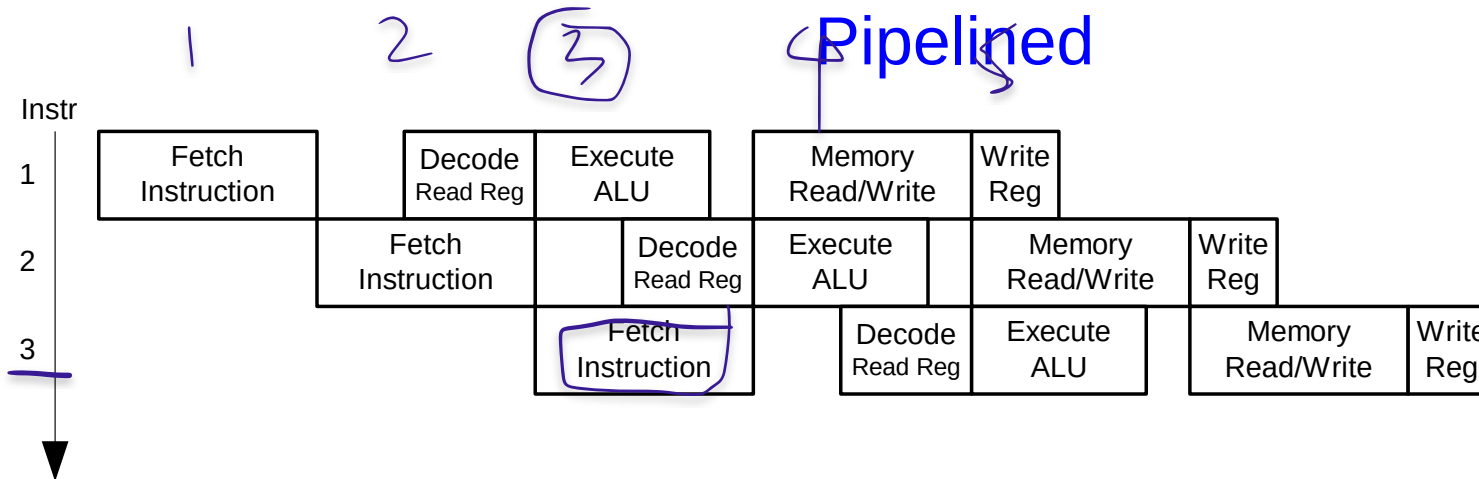
Instr	<u>Instr fetch</u>	<u>Register read</u>	<u>ALU op</u>	<u>Memory access</u>	<u>Register write</u>	Total time
lw	200ps ✓	100 ps	200ps	200ps ✓	100 ps	800ps ✓
sw	200ps ✓	100 ps	200ps	200ps ✓	_____	700ps
R-format	200ps ✓	100 ps	200ps	✗	100 ps	600ps
beq	200ps ✓	100 ps	200ps			500ps

SINGLE-CYCLE VS. PIPELINED PERFORMANCE

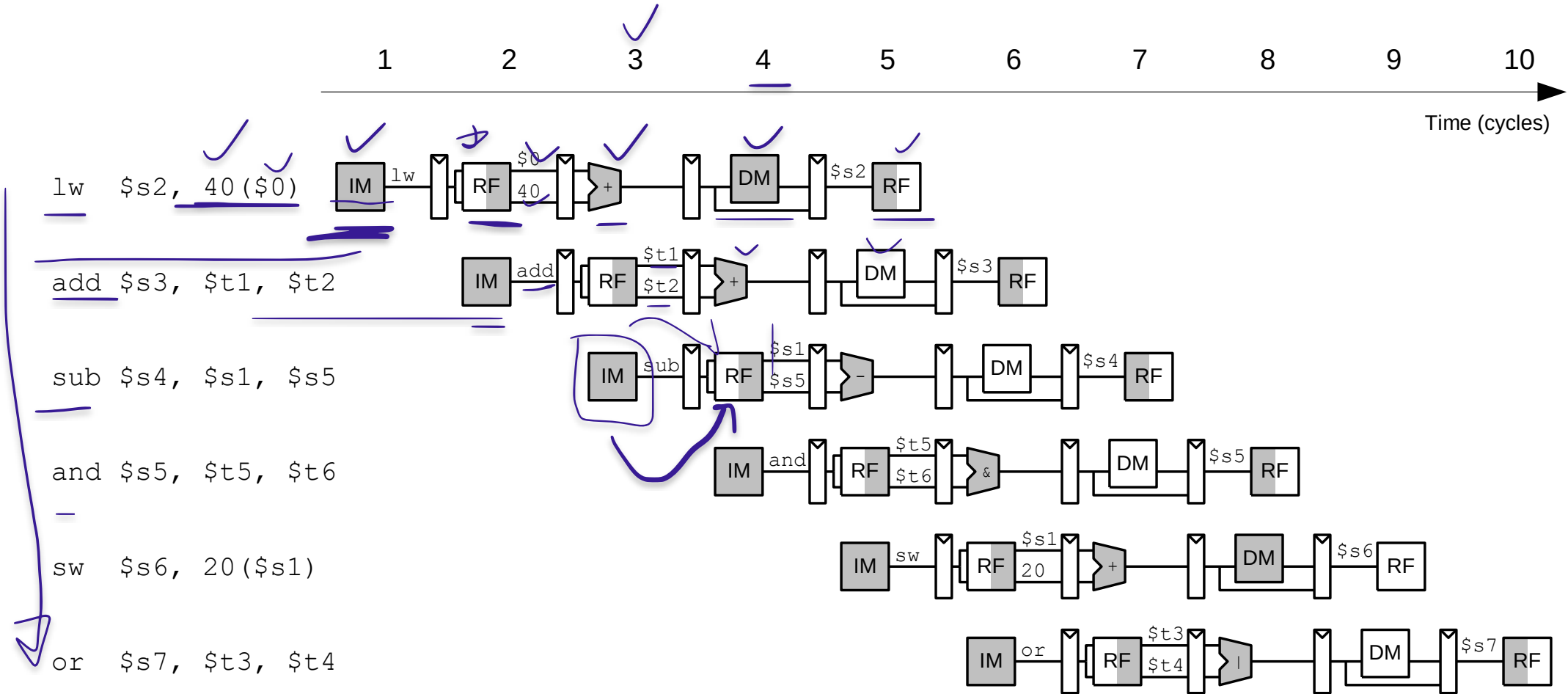
Single-Cycle



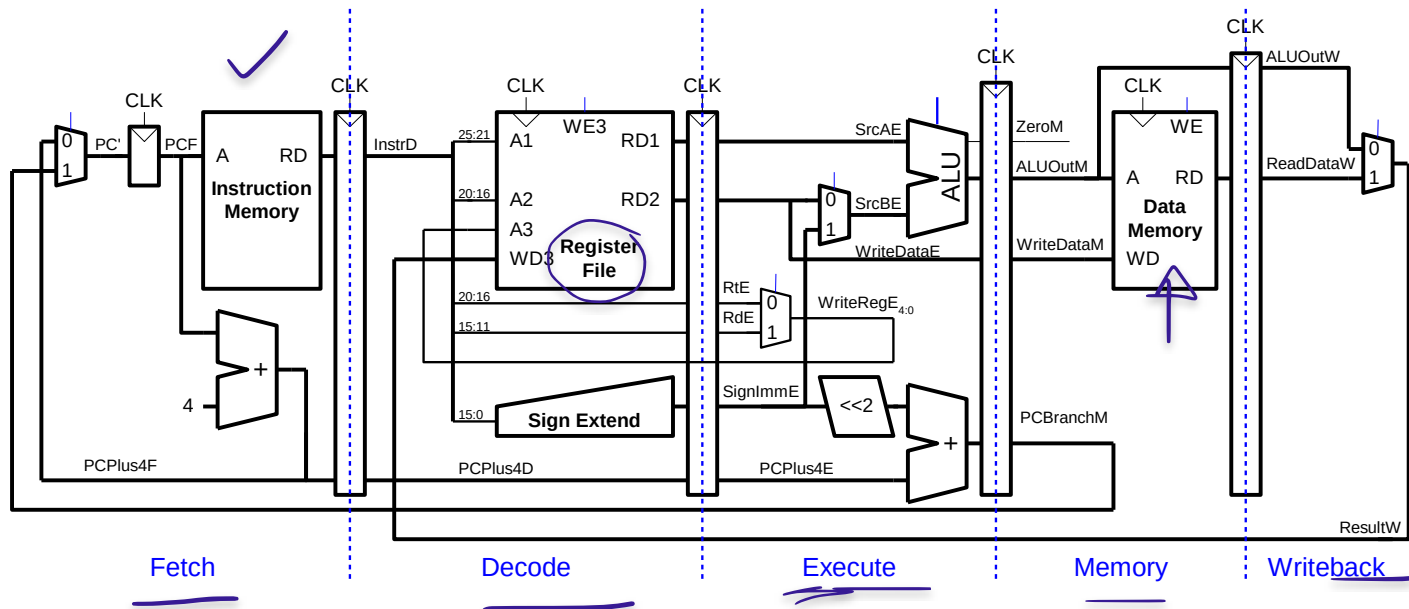
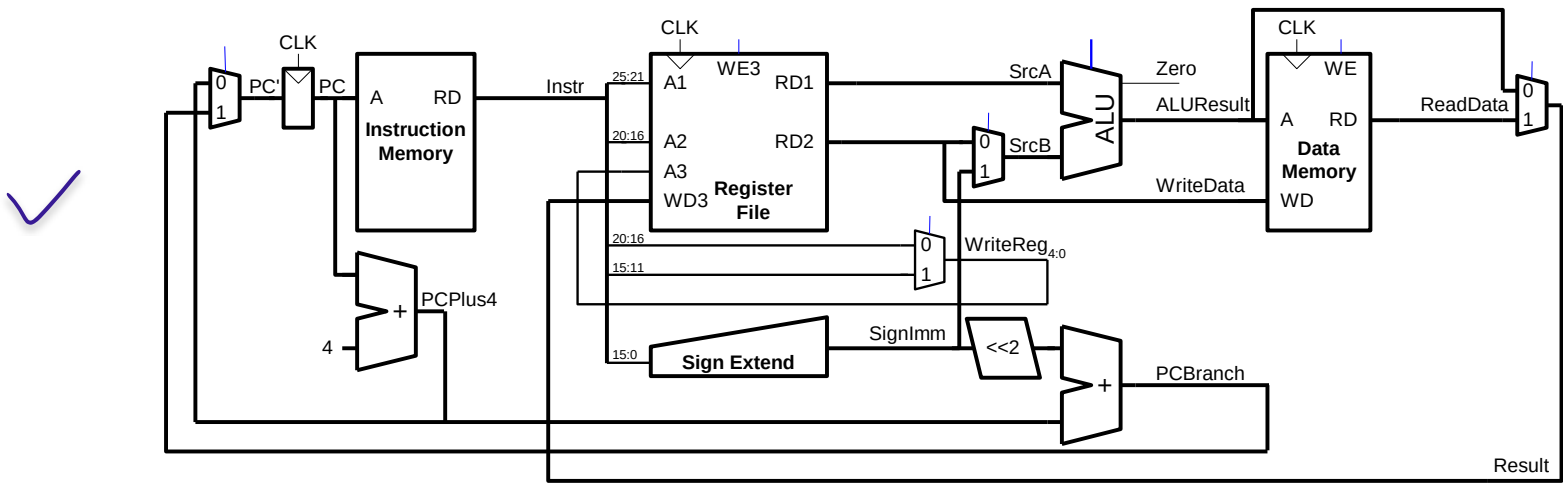
Pipelined



PIPELINING ABSTRACTION

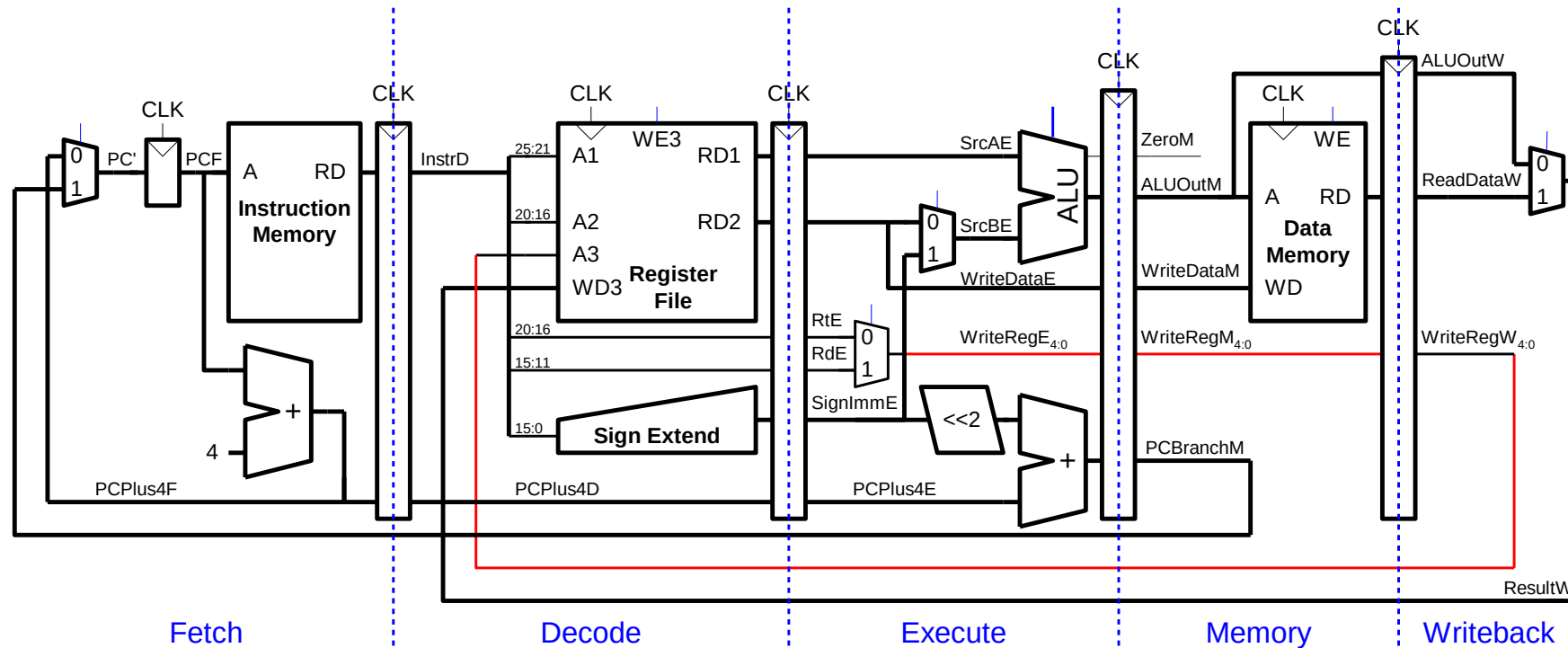


SINGLE-CYCLE AND PIPELINED DATAPATH

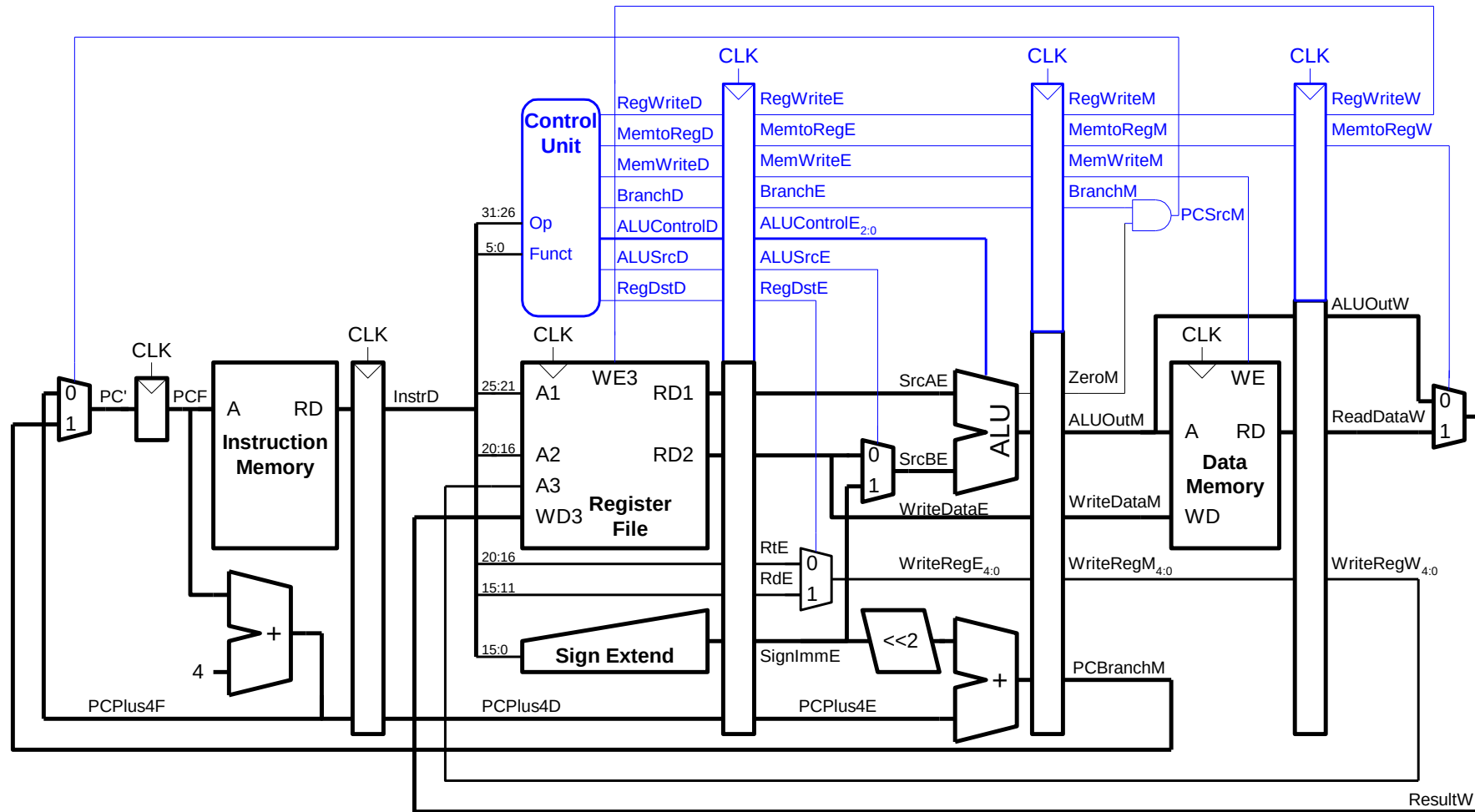


CORRECTED PIPELINED DATAPATH

- WriteReg must arrive at the same time as Result



PIPELINED CONTROL



Same control unit as single-cycle processor

Control delayed to proper pipeline stage

PIPELINE HAZARDS

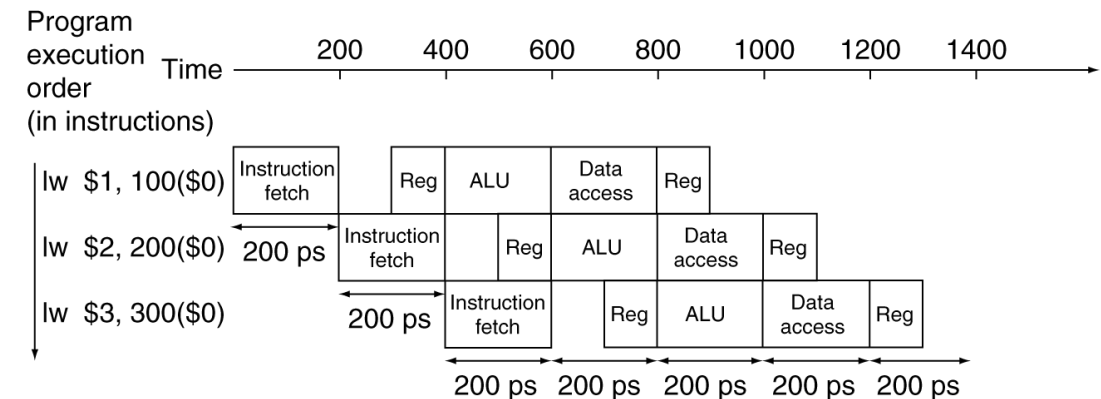
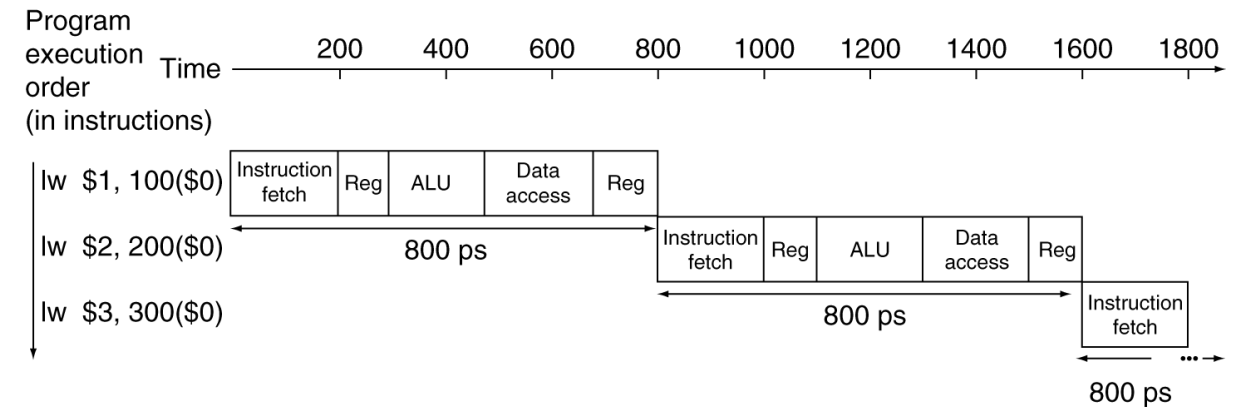
PIPELINE HAZARD

- A pipeline hazard in computer architecture refers to a situation that prevents the next instruction in the instruction pipeline from executing during its designated clock cycle, causing delays and reducing the efficiency of pipelining.
- Types of hazards:
 - **Structural hazard:** The hardware can not support the combinations of instructions in the same clock cycle
 - **Data hazard:** register value not written back to register file yet
 - **Control hazard:** next instruction not decided yet (caused by branches)

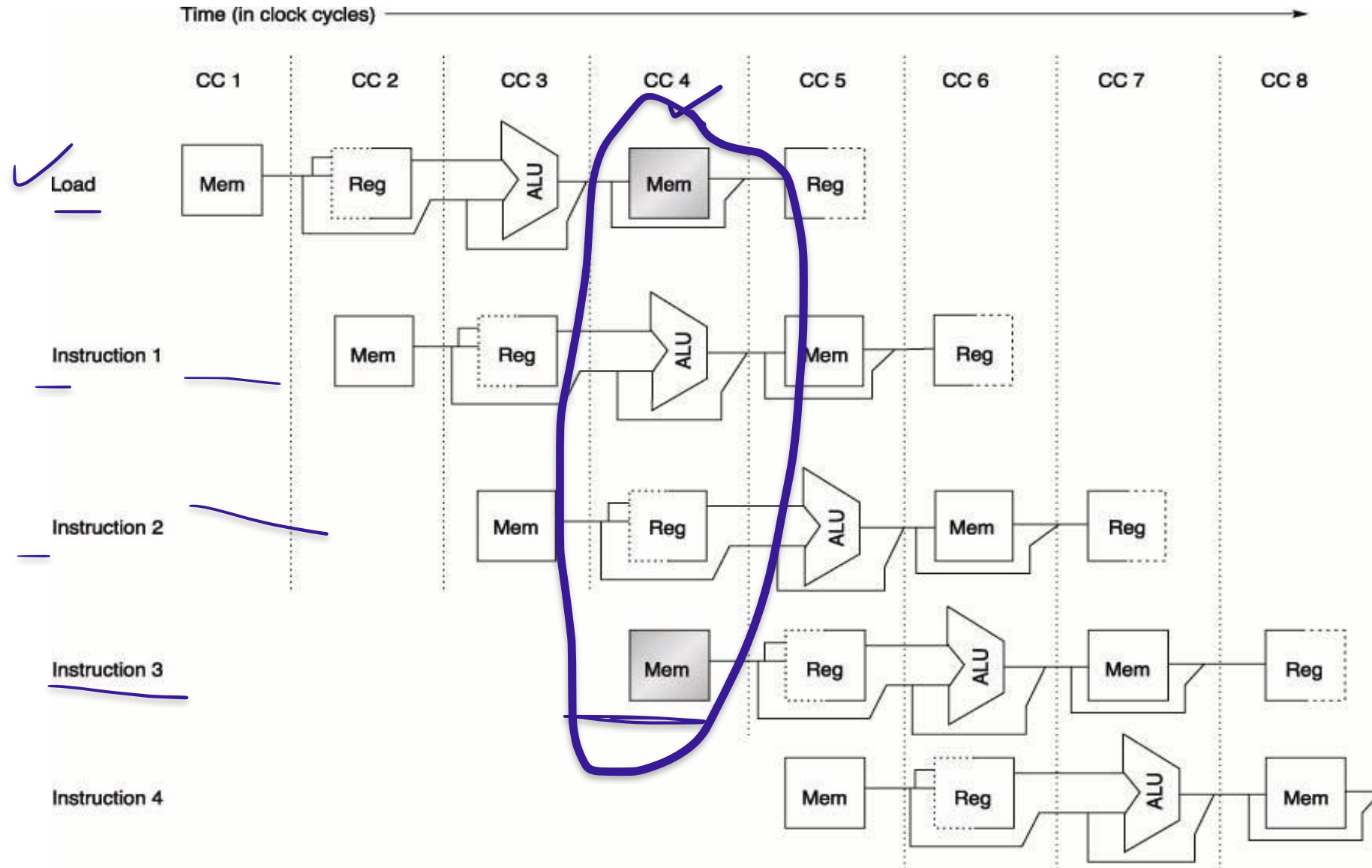
$a = b + c;$
 $e = a - k;$ ✓

STRUCTURAL HAZARD

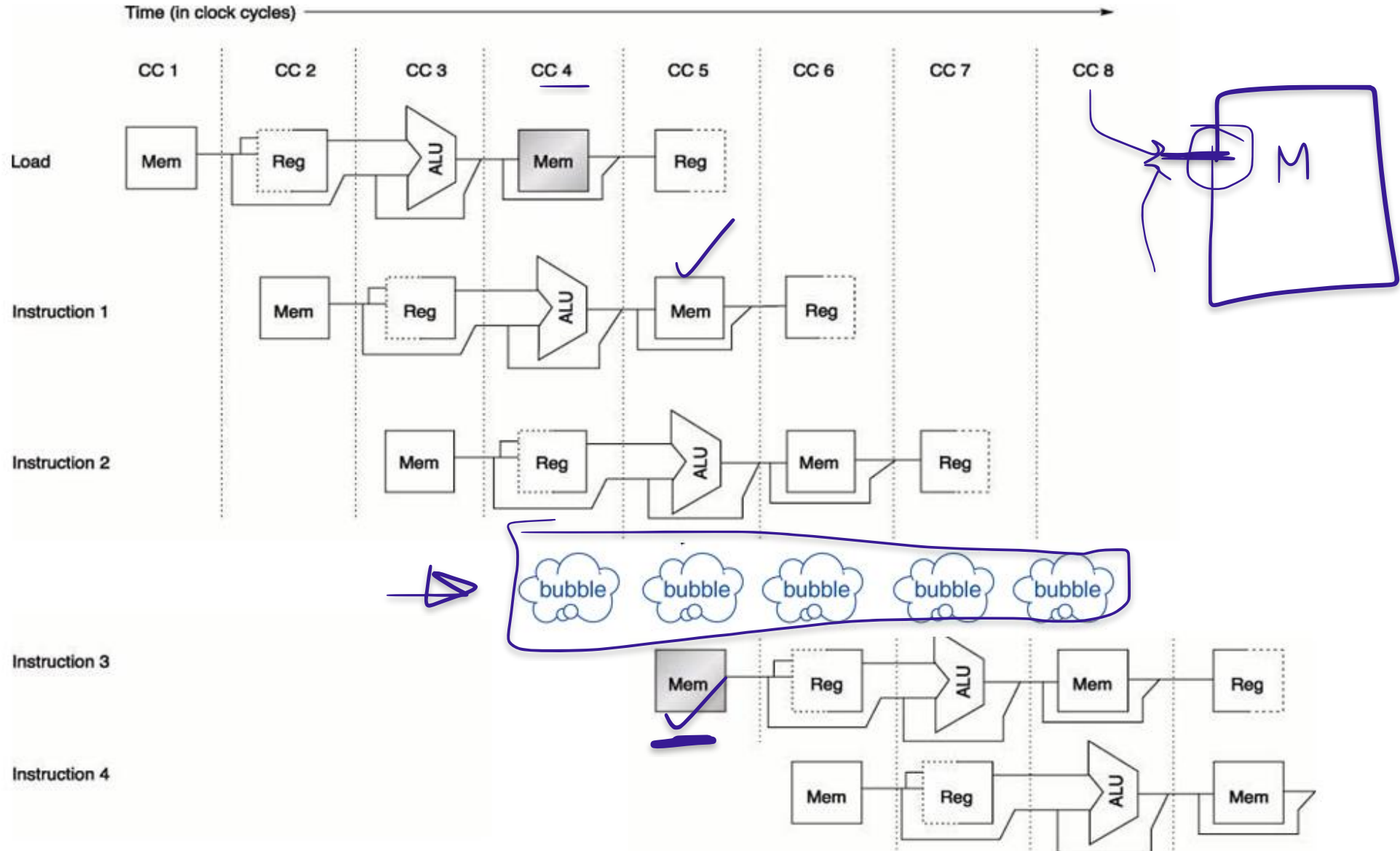
- Conflict for use of a resource
- In MIPS pipeline with a single memory
 - Load/store requires data access
 - Instruction fetch would have to *stall* for that cycle
 - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
 - Or separate instruction/data caches
 - In general, resources need to be replicated to avoid structural hazard and stall



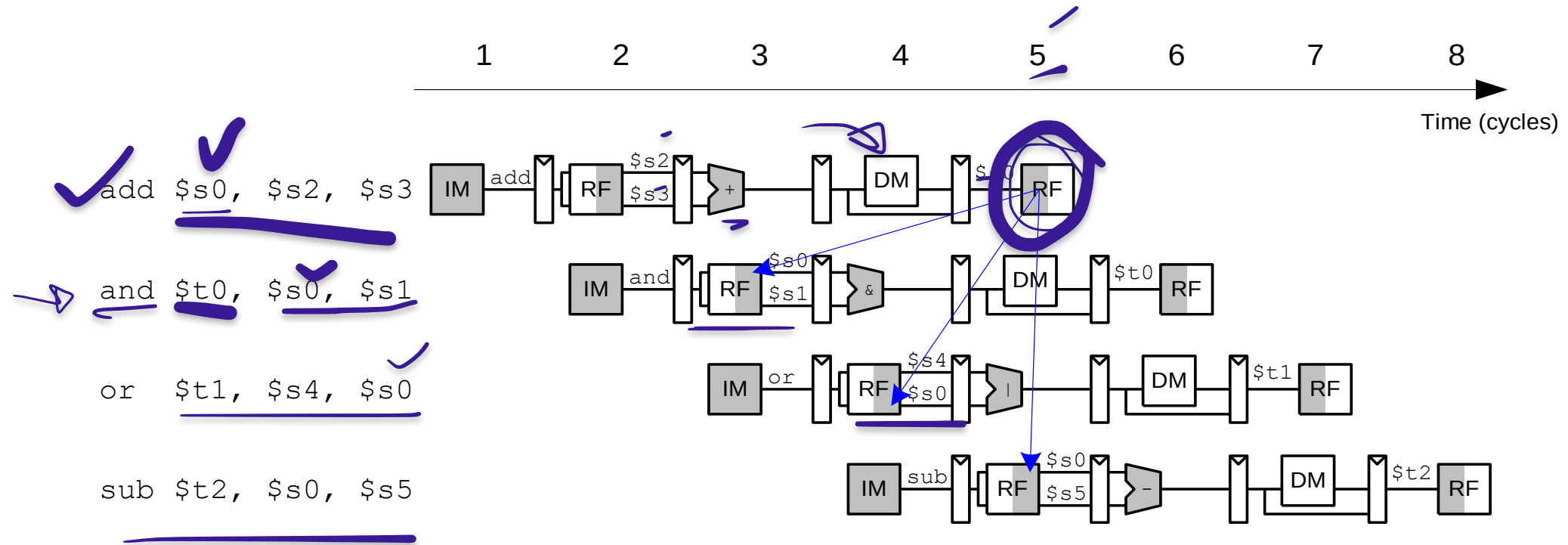
STRUCTURAL HAZARD



STRUCTURAL HAZARD



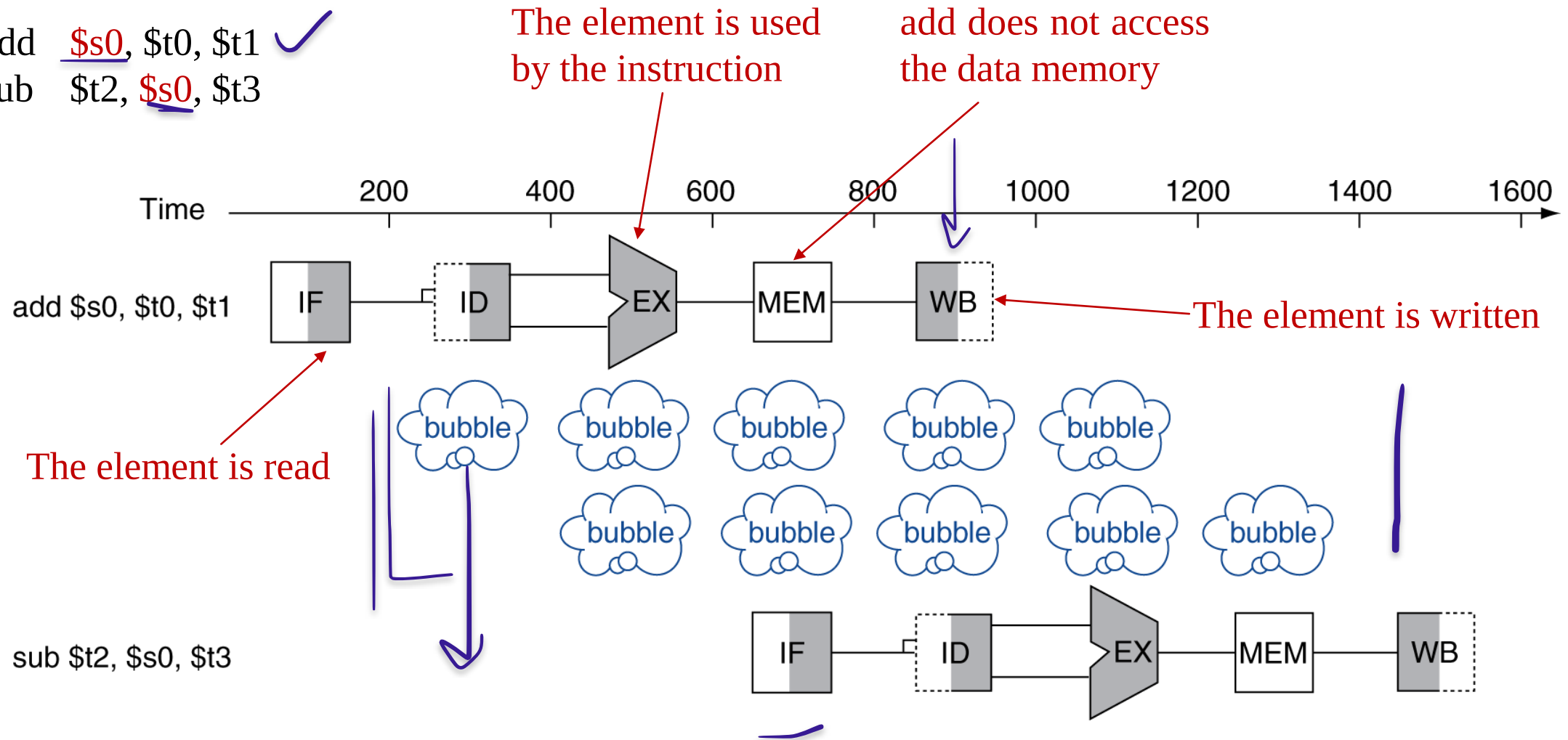
DATA HAZARD



DATA HAZARDS

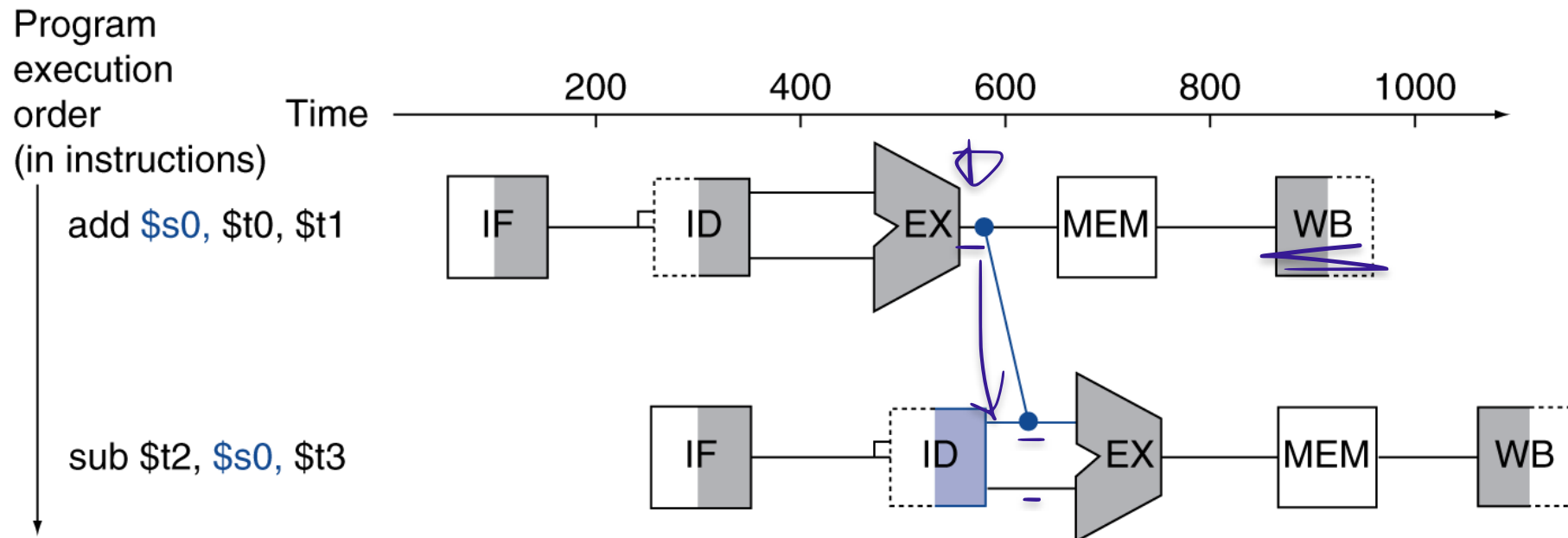
- An instruction depends on completion of data access by a previous instruction

add \$s0, \$t0, \$t1 ✓
sub \$t2, \$s0, \$t3



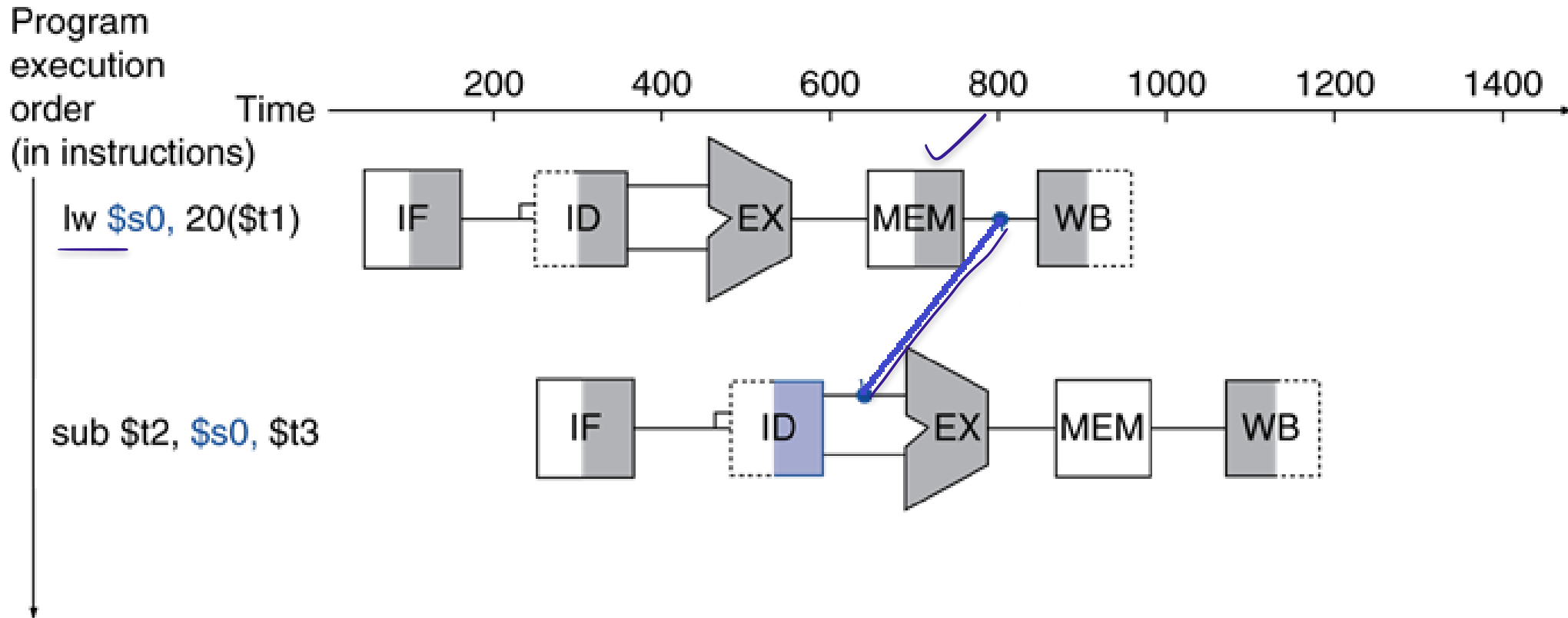
FORWARDING (BYPASSING)

- Use result when it is computed
- Don't wait for it to be stored in a register
- Requires extra connections in the datapath



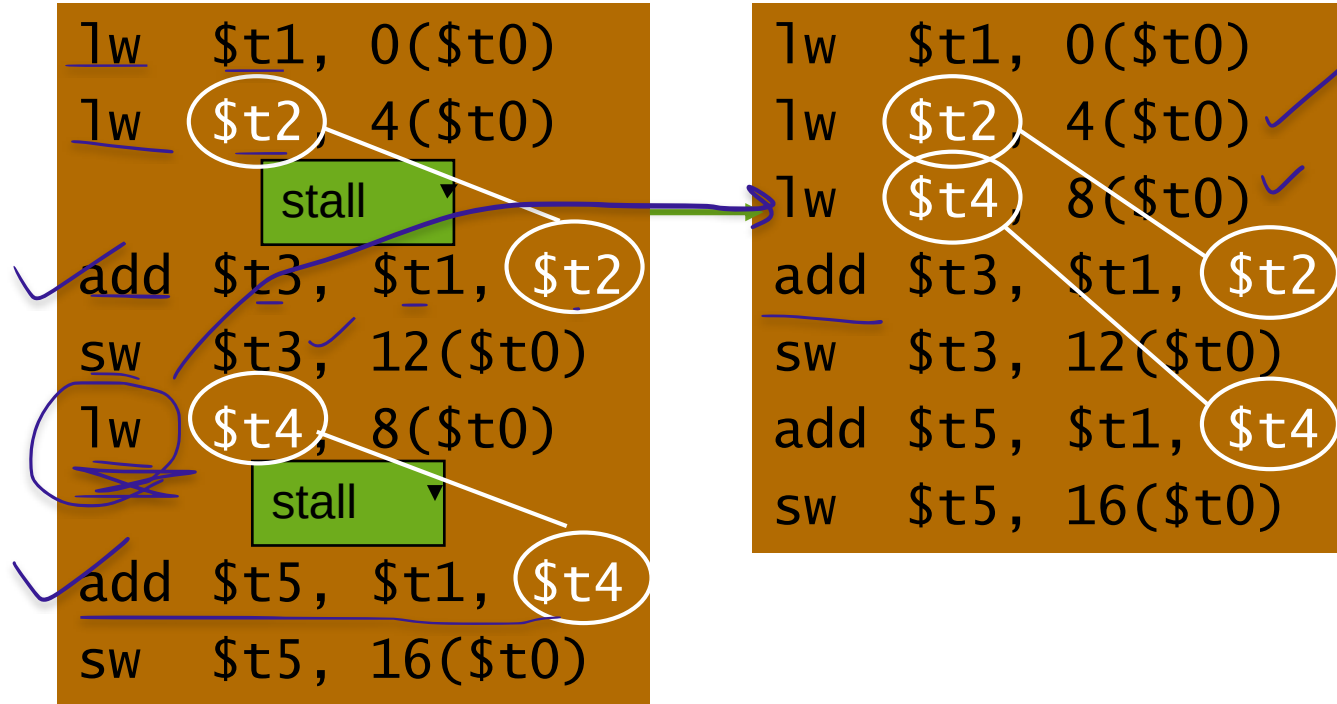
LOAD-USE DATA HAZARD

- Can't always avoid stalls by forwarding
- If value not computed when needed
- Can't forward backward in time!

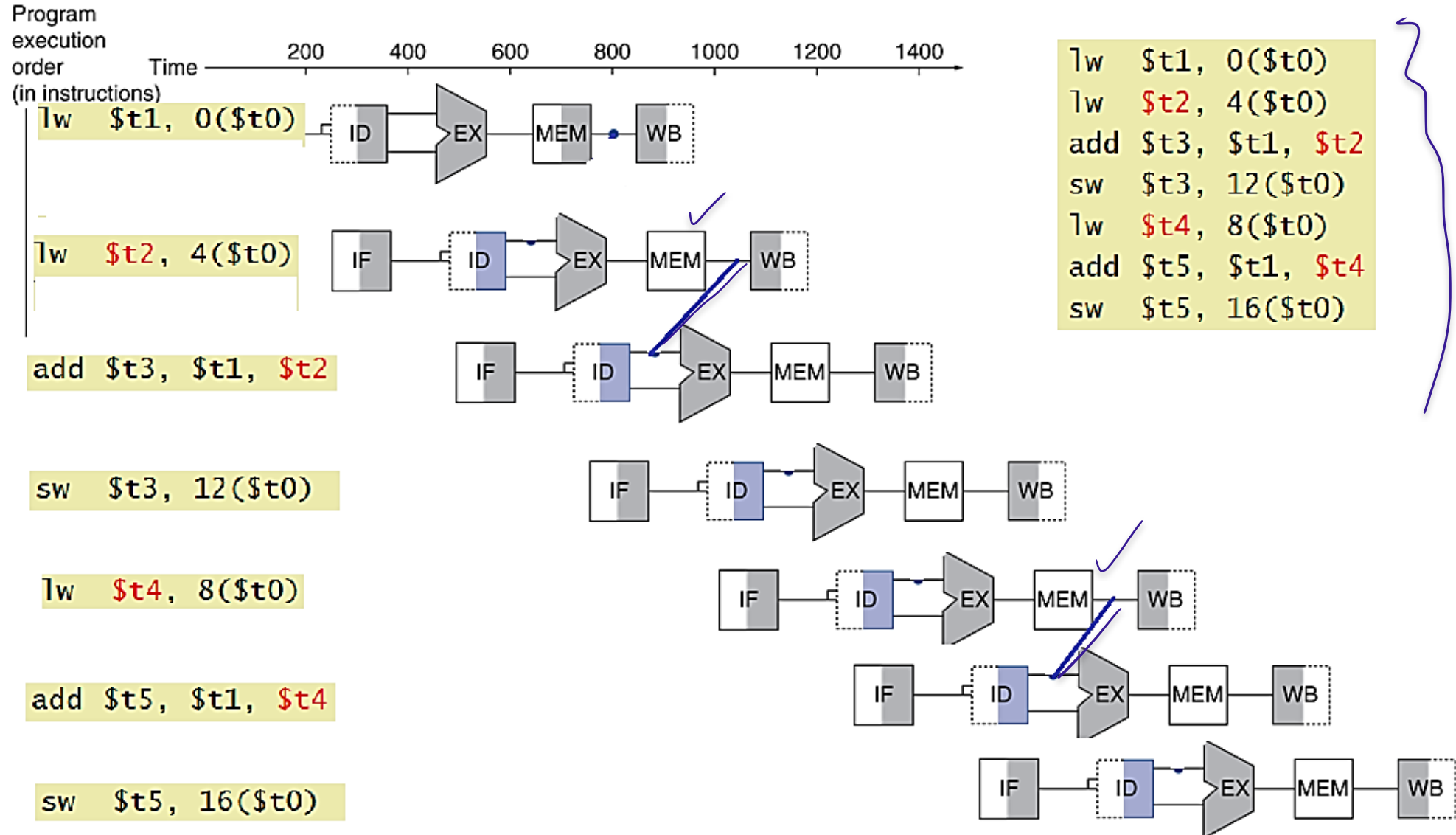


CODE SCHEDULING TO AVOID STALLS

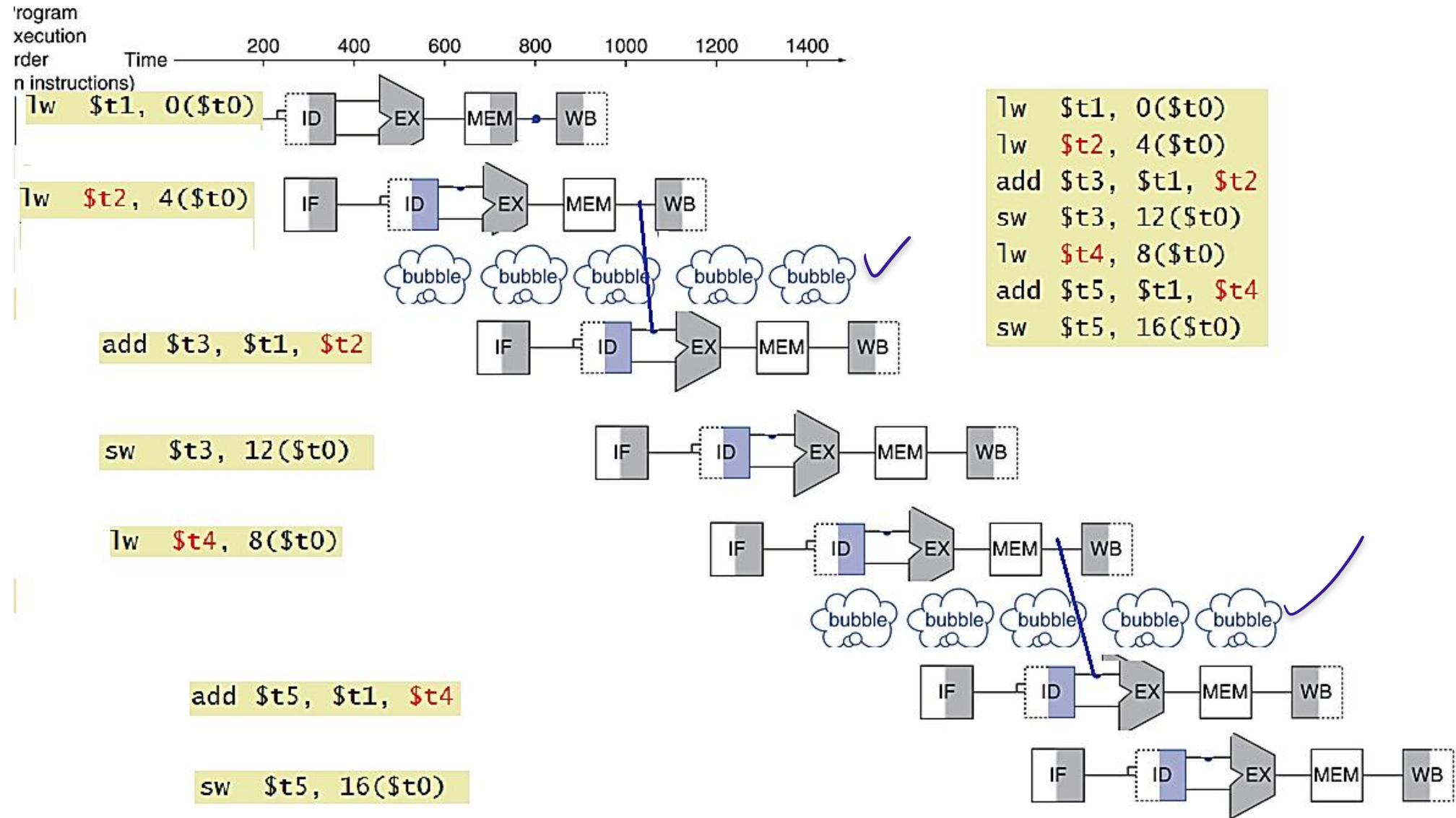
- Reorder code to avoid use of load result in the next instruction
- C code for $A = B + E$; $C = B + F$;



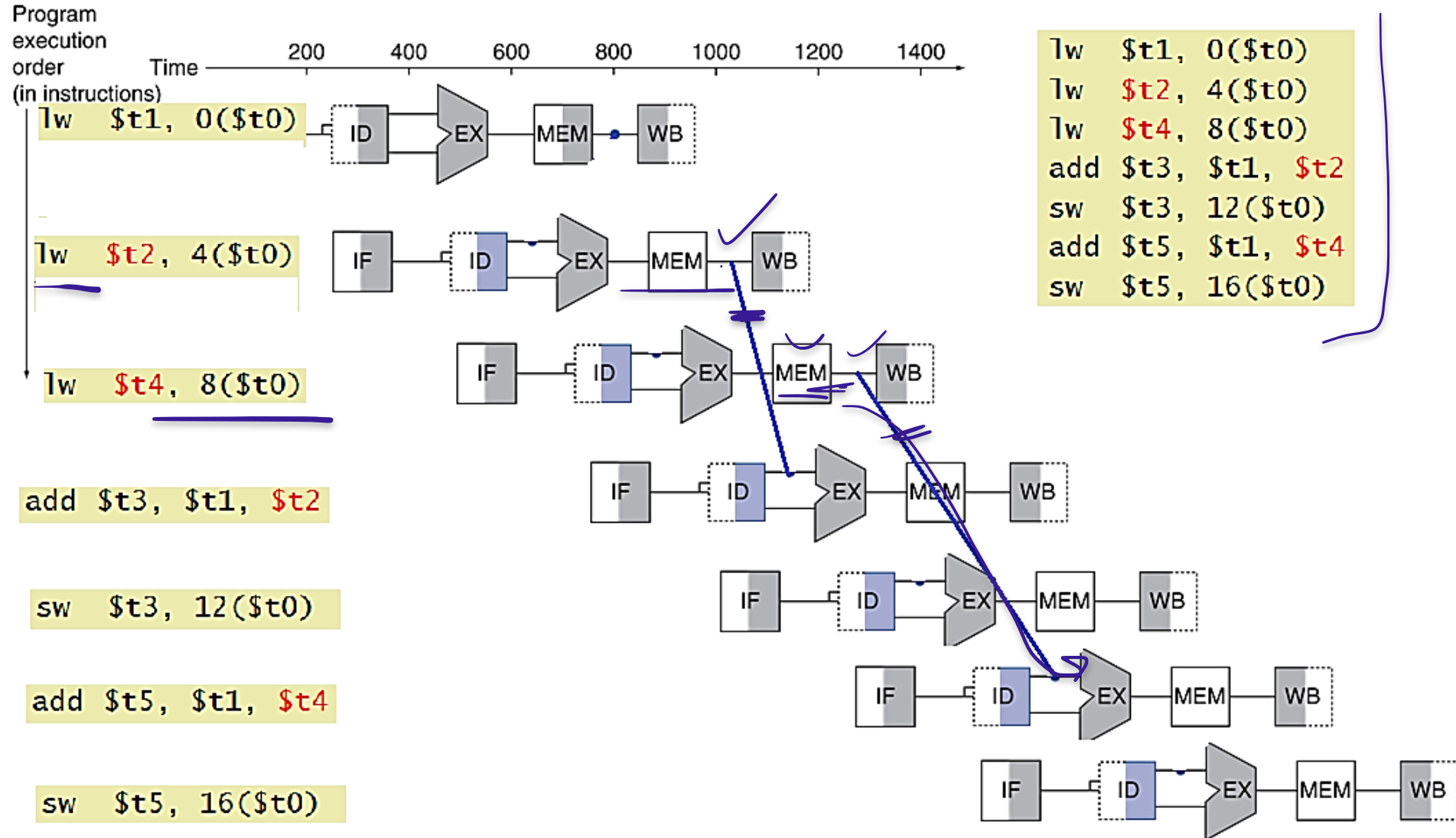
CODE SCHEDULING TO AVOID STALLS



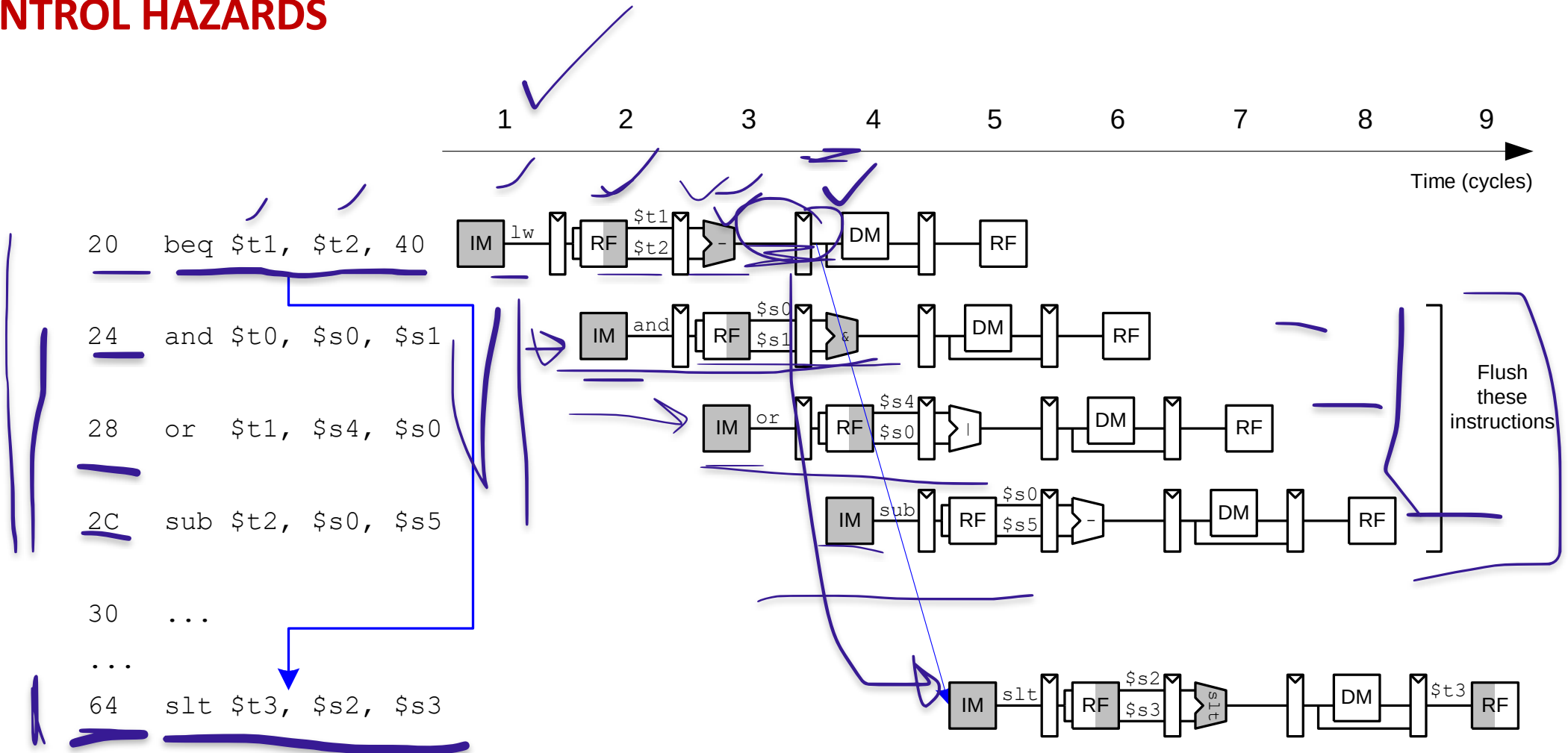
CODE SCHEDULING TO AVOID STALLS



CODE SCHEDULING TO AVOID STALLS



CONTROL HAZARDS



CONTROL HAZARDS

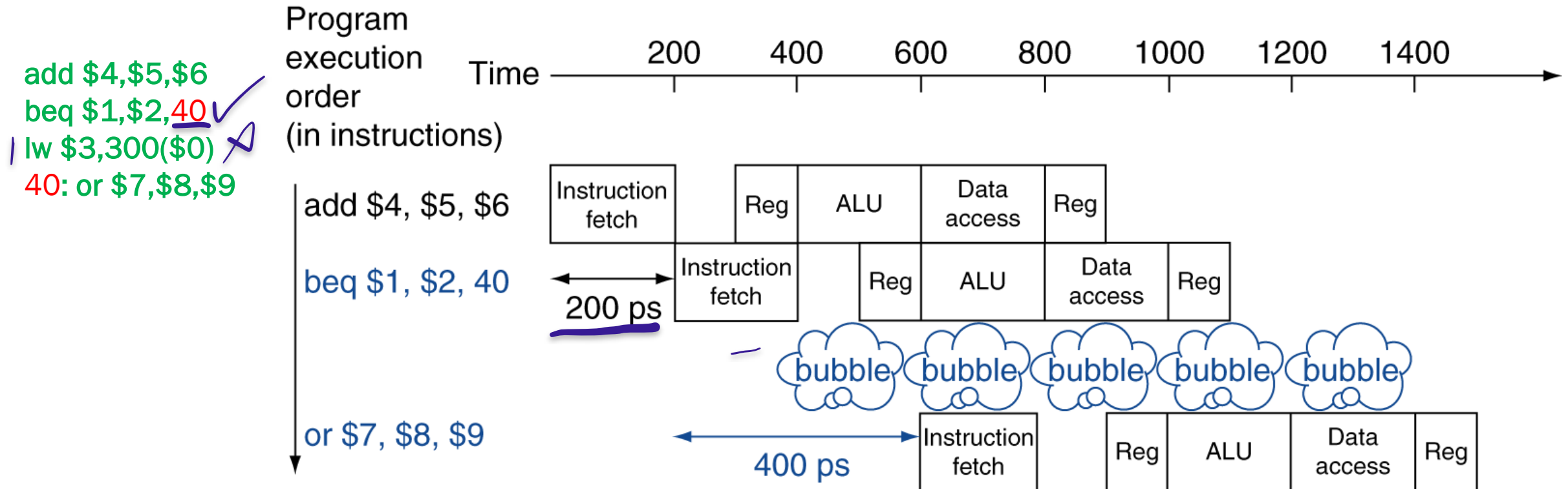
- beq:
 - branch is not determined until the fourth stage of the pipeline
 - Instructions after the branch are fetched before branch occurs
 - These instructions must be flushed if the branch happens
- Branch misprediction penalty
 - number of instruction flushed when branch is taken
 - May be reduced by determining branch earlier

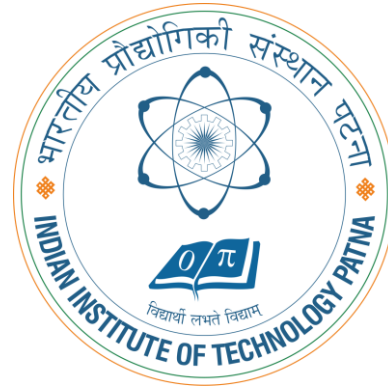
CONTROL HAZARDS

- Branch determines flow of control
 - Fetching next instruction depends on branch outcome
 - Pipeline can't always fetch correct instruction
 - Still working on ID stage of branch
- In MIPS pipeline
 - Need to compare registers and compute target early in the pipeline
 - Add hardware to do it in ID stage

STALL ON BRANCH

- Wait until branch outcome determined (during Decode stage) before fetching next instruction





THANK YOU!